In [38]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

In [39]:
```
# Load dataset
df = pd.read_csv("/content/Gold Price (2013-2023).csv")

# Drop Date column
if 'Date' in df.columns:
    df = df.drop(columns=['Date'])

# Clean numeric columns
def clean_numeric(value):
    value = str(value)
    value = value.replace(',', '')
    value = value.replace('%', '')
    value = value.replace('K', '000')
    value = value.replace('M', '000000')
    return value

for col in df.columns:
    df[col] = df[col].apply(clean_numeric)
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Drop missing values
df = df.dropna()

print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (2578, 6)
```

Out[39]:

	Price	Open	High	Low	Vol.	Change %
0	1826.2	1821.8	1832.4	1819.8	107.50	0.01
1	1826.0	1812.3	1827.3	1811.2	105.99	0.56
2	1815.8	1822.4	1822.8	1804.2	118.08	-0.40
3	1823.1	1808.2	1841.9	1808.0	159.62	0.74
5	1804.2	1801.0	1812.2	1798.9	105.46	0.50

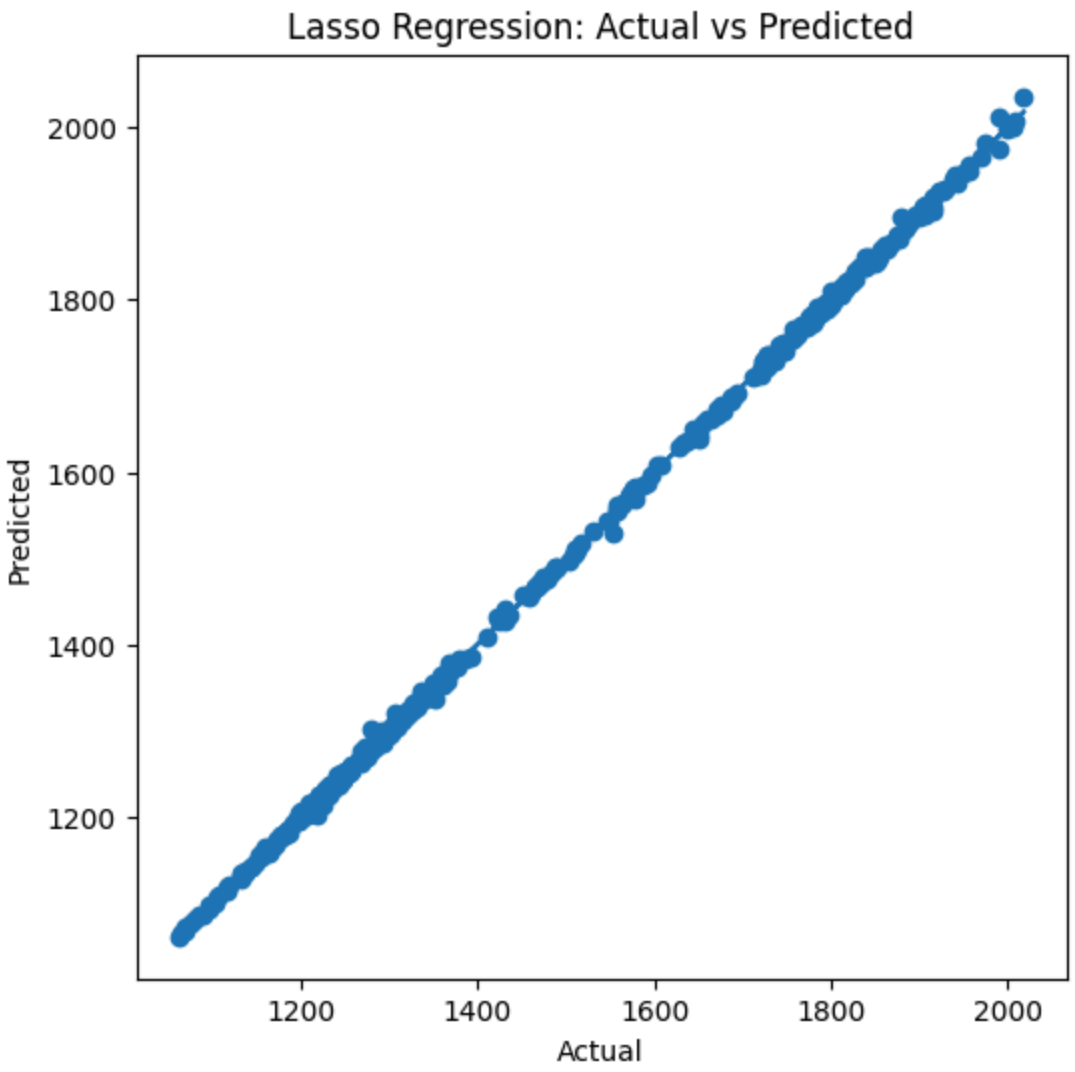
In [40]:
```
y = df['Price']
X = df.drop(columns=['Price'])

# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [41]:
```
def evaluate_model(name, y_test, y_pred):
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print(f"\n{name} Metrics")
    print("MAE:", mae)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("R2:", r2)

    # Actual vs Predicted Plot
    plt.figure(figsize=(6,6))
    plt.scatter(y_test, y_pred)
    plt.plot(y_test, y_test)
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"{name}: Actual vs Predicted")
    plt.show()
```

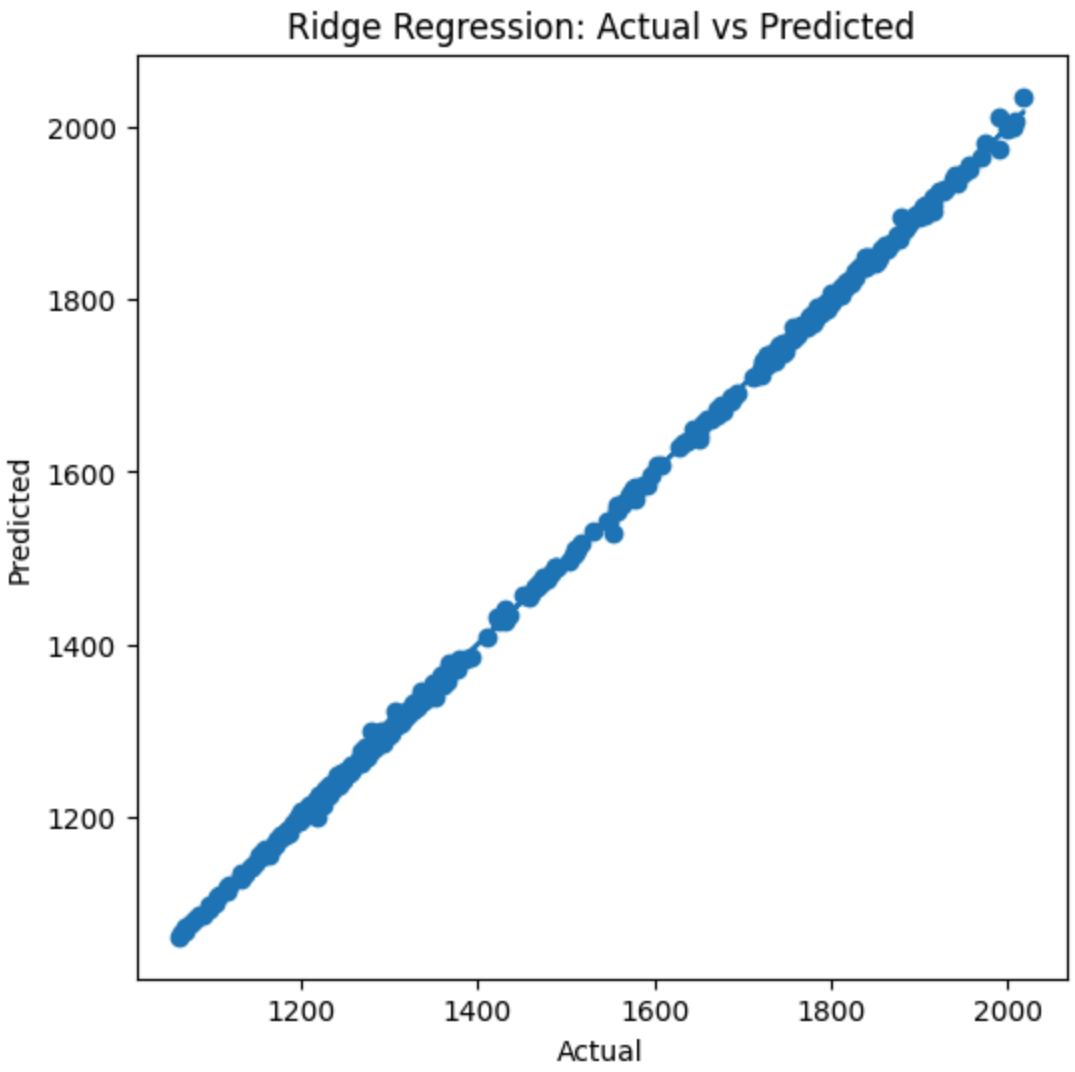
In [43]:
```
lasso_params = {'alpha': [0.001, 0.01, 0.1, 1, 10]}
lasso_grid = GridSearchCV(Lasso(max_iter=10000), lasso_params, cv=5)

lasso_grid.fit(X_train, y_train)

print("Best Lasso Parameters:", lasso_grid.best_params_)

best_lasso = lasso_grid.best_estimator_
y_pred_lasso = best_lasso.predict(X_test)

evaluate_model("Lasso Regression", y_test, y_pred_lasso)
```

```
Best Lasso Parameters: {'alpha': 0.001}

Lasso Regression Metrics
MAE: 2.8482555142839425
MSE: 17.251599386950755
RMSE: 4.153504470558658
R2: 0.9997466297225464
```



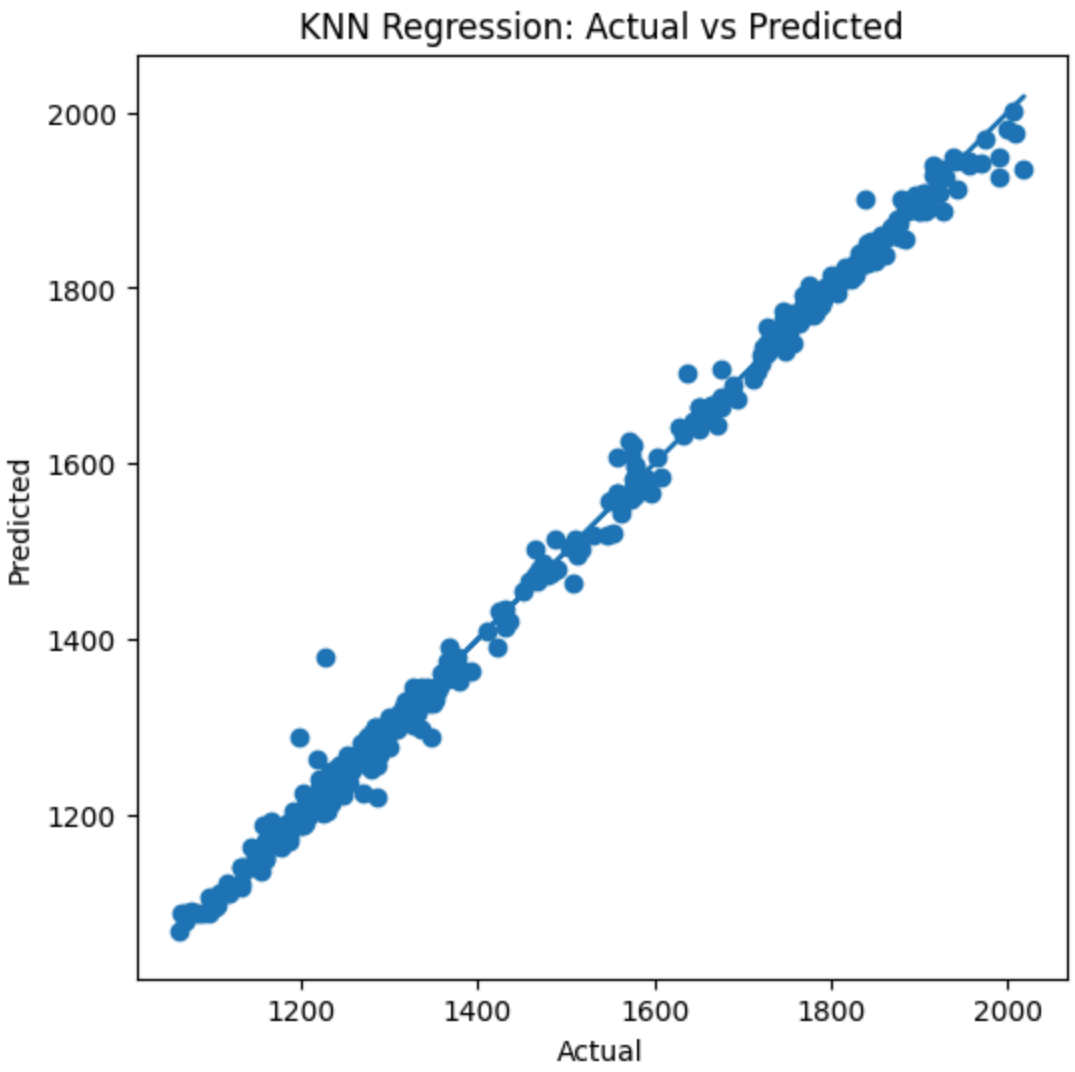
In [44]:
```
ridge_params = {'alpha': [0.001, 0.01, 0.1, 1, 10]}
ridge_grid = GridSearchCV(Ridge(), ridge_params, cv=5)

ridge_grid.fit(X_train, y_train)

print("Best Ridge Parameters:", ridge_grid.best_params_)

best_ridge = ridge_grid.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

evaluate_model("Ridge Regression", y_test, y_pred_ridge)
```

```
Best Ridge Parameters: {'alpha': 0.001}

Ridge Regression Metrics
MAE: 2.8563152622484984
MSE: 17.24649727065702
RMSE: 4.152890230990583
R2: 0.9997467046561564
```



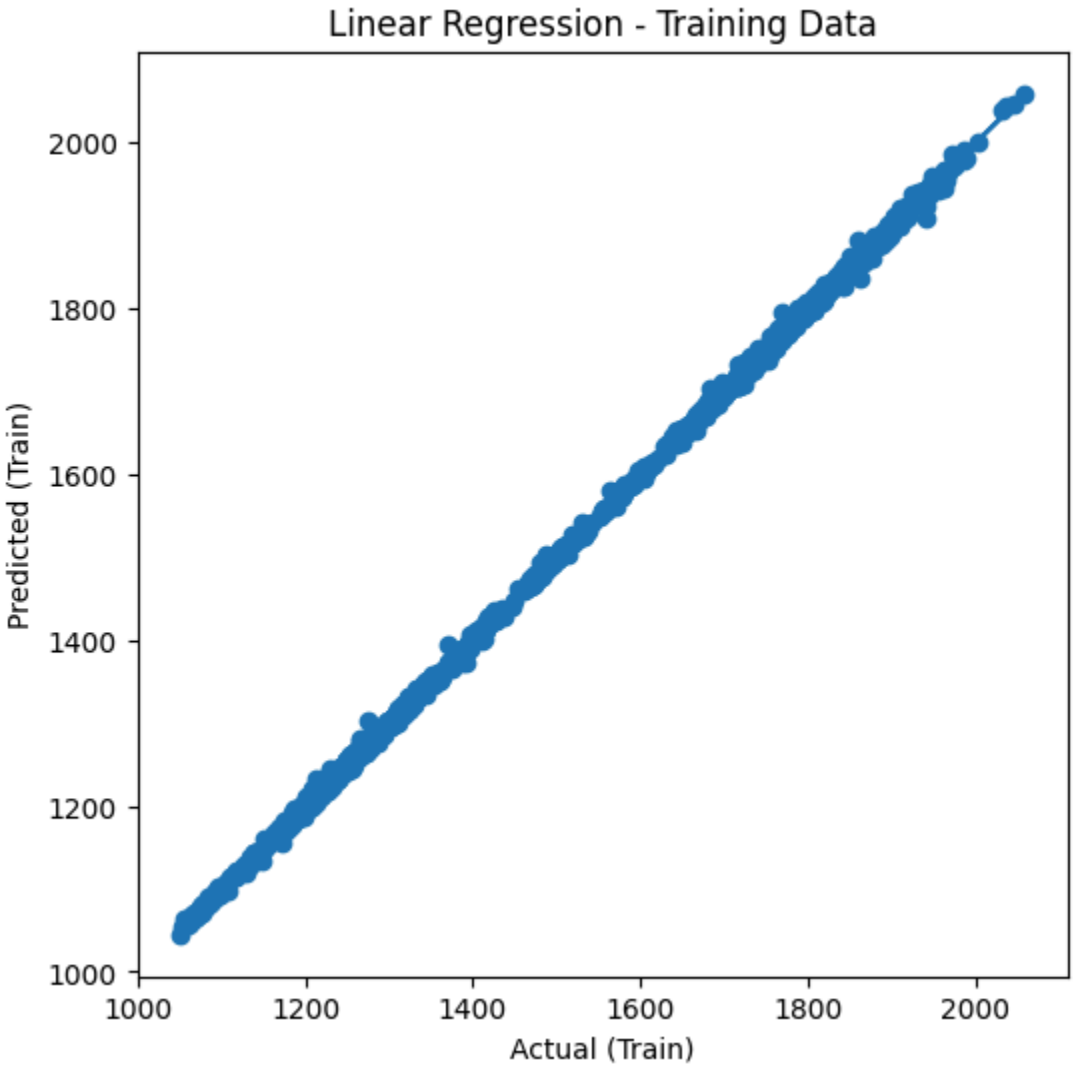
In [45]:
```
knn_params = {'n_neighbors': [3, 5, 7, 9, 11]}
knn_grid = GridSearchCV(KNeighborsRegressor(), knn_params, cv=5)

knn_grid.fit(X_train, y_train)

print("Best KNN Parameters:", knn_grid.best_params_)

best_knn = knn_grid.best_estimator_
y_pred_knn = best_knn.predict(X_test)

evaluate_model("KNN Regression", y_test, y_pred_knn)
```

```
Best KNN Parameters: {'n_neighbors': 3}

KNN Regression Metrics
MAE: 10.012790697674422
MSE: 267.3016537467701
RMSE: 16.349362487472412
R2: 0.996074202011388
```



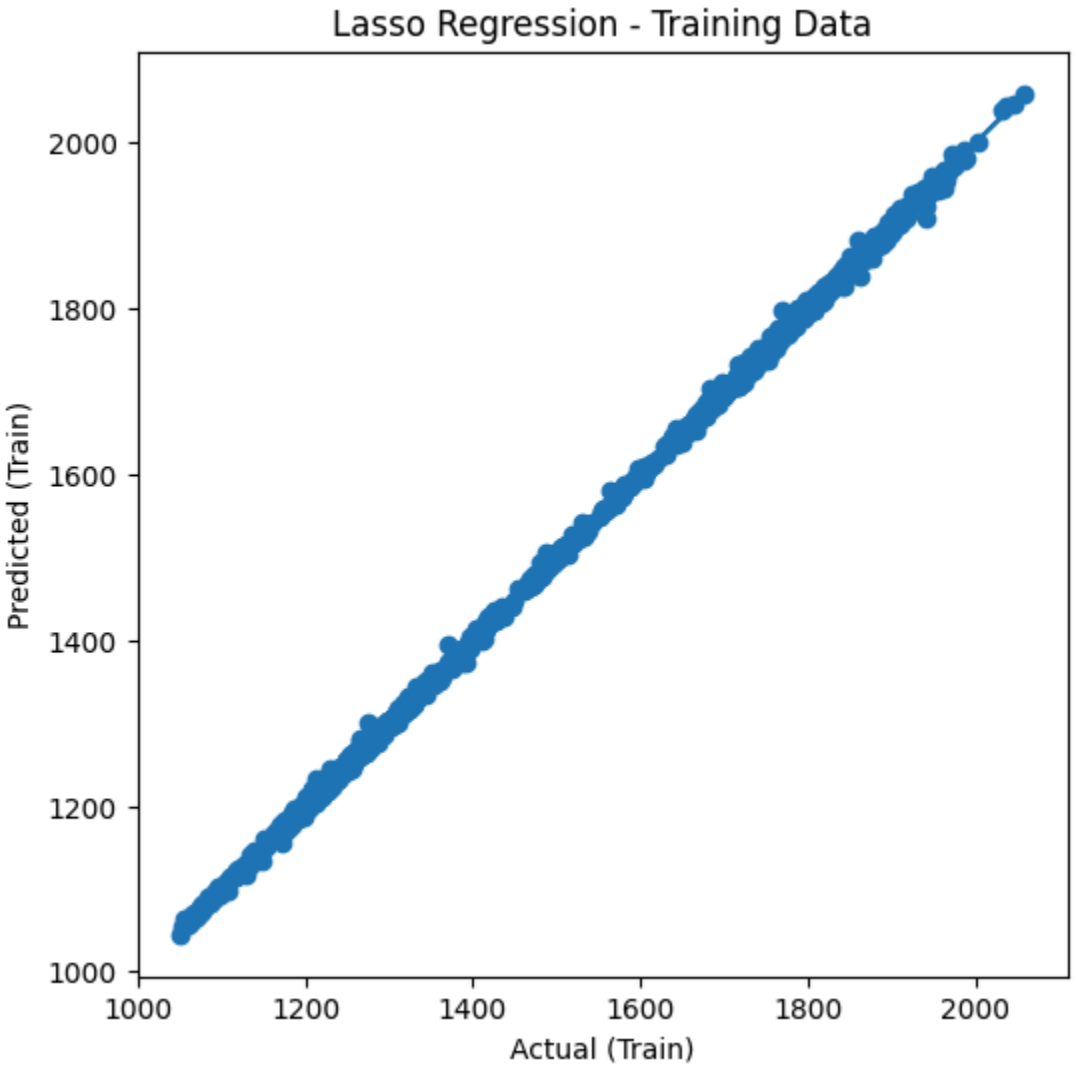
In [47]:
```python
# Predict on training data
y_pred_train_lr = lr.predict(X_train)

# Training Metrics
print("Linear Regression - TRAINING METRICS")
print("MAE:", mean_absolute_error(y_train, y_pred_train_lr))
print("MSE:", mean_squared_error(y_train, y_pred_train_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_train_lr)))
print("R2:", r2_score(y_train, y_pred_train_lr))

# Plot Training Actual vs Predicted
plt.figure(figsize=(6,6))
plt.scatter(y_train, y_pred_train_lr)
plt.plot(y_train, y_train)
plt.xlabel("Actual (Train)")
plt.ylabel("Predicted (Train)")
plt.title("Linear Regression - Training Data")
plt.show()
```

```
Linear Regression - TRAINING METRICS
MAE: 2.975285866551137
MSE: 18.096190680154823
RMSE: 4.253961762892895
R2: 0.999724324883189
```



In [48]:
```python
y_pred_train_lasso = best_lasso.predict(X_train)

print("Lasso Regression - TRAINING METRICS")
print("MAE:", mean_absolute_error(y_train, y_pred_train_lasso))
print("MSE:", mean_squared_error(y_train, y_pred_train_lasso))
print("RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_train_lasso)))
print("R2:", r2_score(y_train, y_pred_train_lasso))

plt.figure(figsize=(6,6))
plt.scatter(y_train, y_pred_train_lasso)
plt.plot(y_train, y_train)
plt.xlabel("Actual (Train)")
plt.ylabel("Predicted (Train)")
plt.title("Lasso Regression - Training Data")
plt.show()
```

```
Lasso Regression - TRAINING METRICS
MAE: 2.9687246573190267
MSE: 18.166067020397374
RMSE: 4.26216693952705
R2: 0.9997232603957176
```

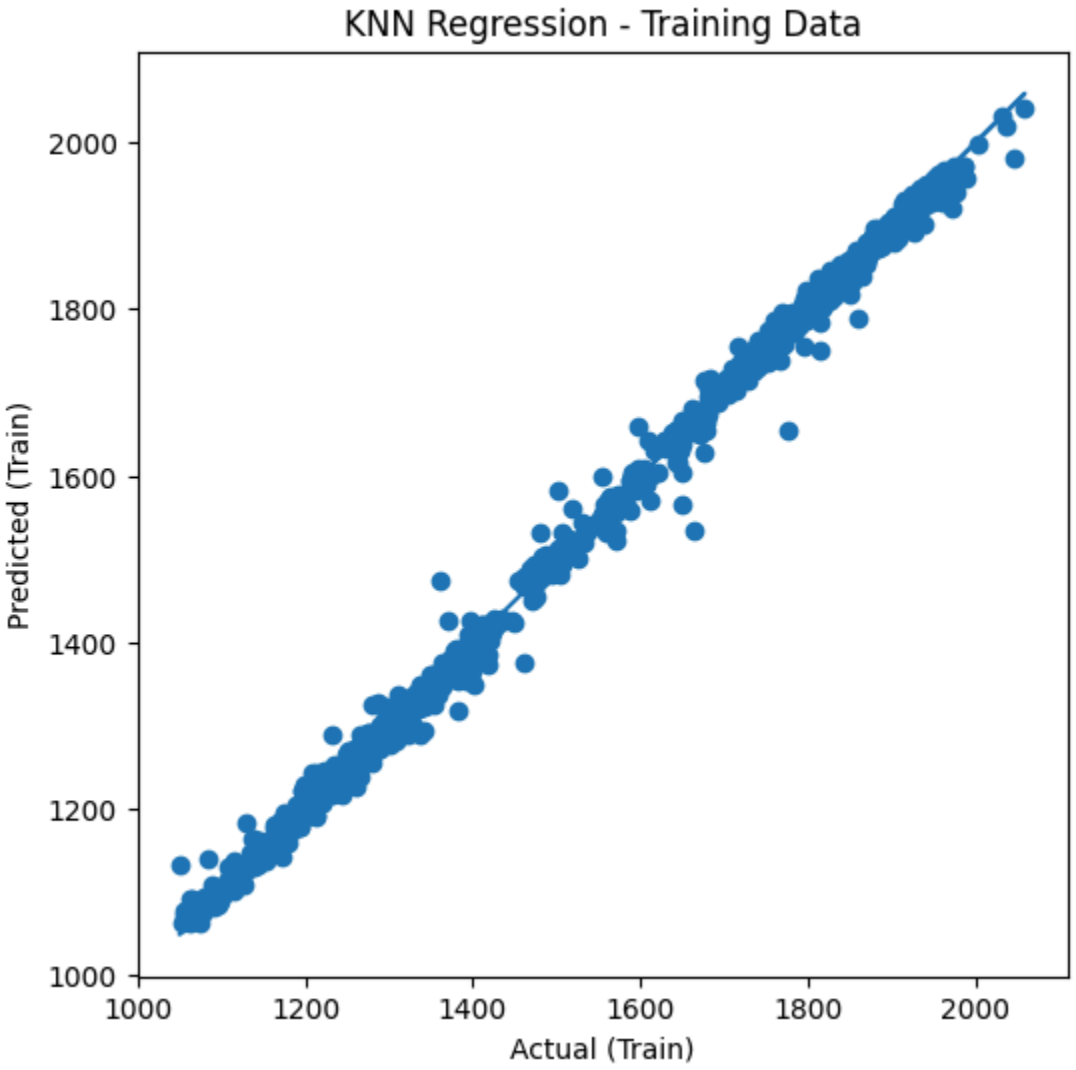


In [49]:
```python
y_pred_train_ridge = best_ridge.predict(X_train)

print("Ridge Regression - TRAINING METRICS")
print("MAE:", mean_absolute_error(y_train, y_pred_train_ridge))
print("MSE:", mean_squared_error(y_train, y_pred_train_ridge))
print("RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_train_ridge)))
print("R2:", r2_score(y_train, y_pred_train_ridge))

plt.figure(figsize=(6,6))
plt.scatter(y_train, y_pred_train_ridge)
plt.plot(y_train, y_train)
plt.xlabel("Actual (Train)")
plt.ylabel("Predicted (Train)")
plt.title("Ridge Regression - Training Data")
plt.show()
```

```
Ridge Regression - TRAINING METRICS
MAE: 2.975188169093759
MSE: 18.096199334244687
RMSE: 4.253962780072798
R2: 0.9997243247513536
```

In [50]:
```python
y_pred_train_knn = best_knn.predict(X_train)

print("KNN Regression - TRAINING METRICS")
print("MAE:", mean_absolute_error(y_train, y_pred_train_knn))
print("MSE:", mean_squared_error(y_train, y_pred_train_knn))
print("RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_train_knn)))
print("R2:", r2_score(y_train, y_pred_train_knn))

plt.figure(figsize=(6,6))
plt.scatter(y_train, y_pred_train_knn)
plt.plot(y_train, y_train)
plt.xlabel("Actual (Train)")
plt.ylabel("Predicted (Train)")
plt.title("KNN Regression - Training Data")
plt.show()
```

```
KNN Regression - TRAINING METRICS
MAE: 7.158713223407693
MSE: 146.9216779825412
RMSE: 12.121125277074782
R2: 0.9977618134415257
```

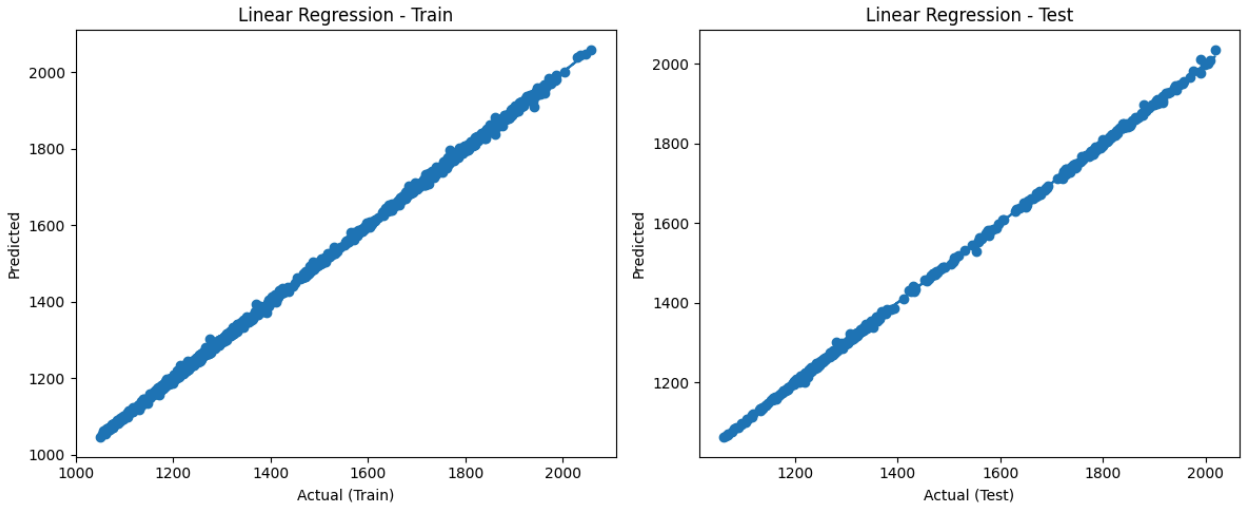


In [51]:
```python
def compare_train_test(model, model_name, X_train, X_test, y_train, y_test):

    # Predictions
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    print(f"\n{'='*40}")
    print(f"{model_name} PERFORMANCE")
    print(f"{'='*40}")

    print("\n--- TRAINING METRICS ---")
    print("MAE:", mean_absolute_error(y_train, y_pred_train))
    print("MSE:", mean_squared_error(y_train, y_pred_train))
    print("RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_train)))
    print("R2:", r2_score(y_train, y_pred_train))
```

```
    print("\n--- TESTING METRICS ---")
    print("MAE:", mean_absolute_error(y_test, y_pred_test))
    print("MSE:", mean_squared_error(y_test, y_pred_test))
    print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_test)))
    print("R2:", r2_score(y_test, y_pred_test))

    # Plot comparison
    plt.figure(figsize=(12,5))

    # Train plot
    plt.subplot(1,2,1)
    plt.scatter(y_train, y_pred_train)
    plt.plot(y_train, y_train)
    plt.xlabel("Actual (Train)")
    plt.ylabel("Predicted")
    plt.title(f"{model_name} - Train")

    # Test plot
    plt.subplot(1,2,2)
    plt.scatter(y_test, y_pred_test)
    plt.plot(y_test, y_test)
    plt.xlabel("Actual (Test)")
    plt.ylabel("Predicted")
    plt.title(f"{model_name} - Test")

    plt.tight_layout()
    plt.show()
```

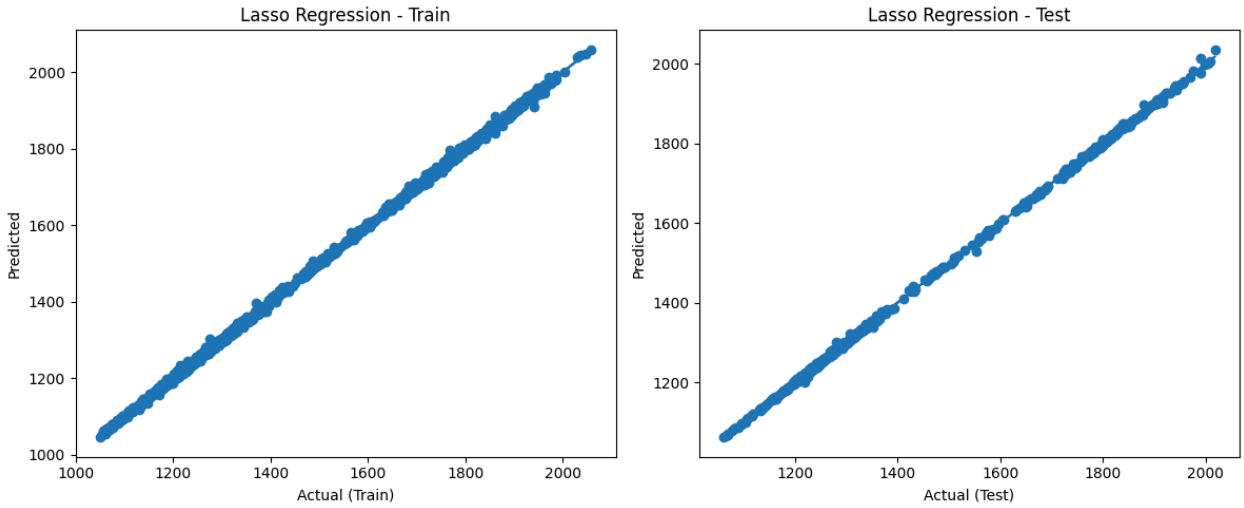
In [52]:
```
compare_train_test(lr, "Linear Regression", X_train, X_test, y_train, y_test)
```

```
========================================
Linear Regression PERFORMANCE
========================================

--- TRAINING METRICS ---
MAE: 2.975285866551137
MSE: 18.096190680154823
RMSE: 4.253961762892895
R2: 0.999724324883189

--- TESTING METRICS ---
MAE: 2.856481620221935
MSE: 17.247517461914303
RMSE: 4.153013058240282
R2: 0.9997466896728417
```



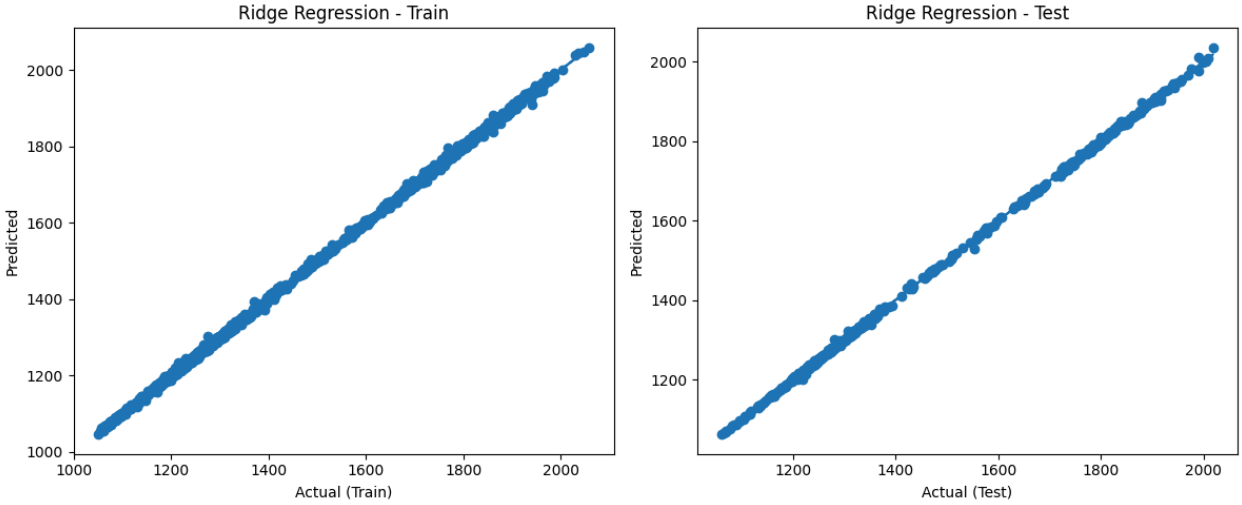
In [53]:
```
compare_train_test(best_lasso, "Lasso Regression", X_train, X_test, y_train, y
```

```
==========================================
Lasso Regression PERFORMANCE
==========================================

--- TRAINING METRICS ---
MAE: 2.9687246573190267
MSE: 18.166067020397374
RMSE: 4.26216693952705
R2: 0.9997232603957176

--- TESTING METRICS ---
MAE: 2.8482555142839425
MSE: 17.251599386950755
RMSE: 4.153504470558658
R2: 0.9997466297225464
```



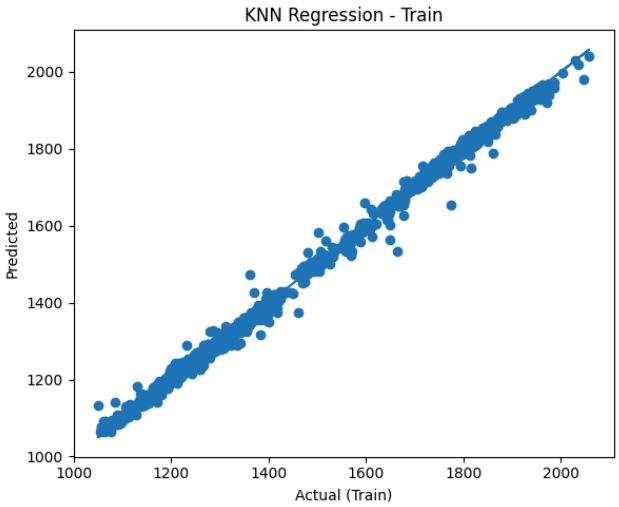
In [54]:
```
compare_train_test(best_ridge, "Ridge Regression", X_train, X_test, y_train, y
```

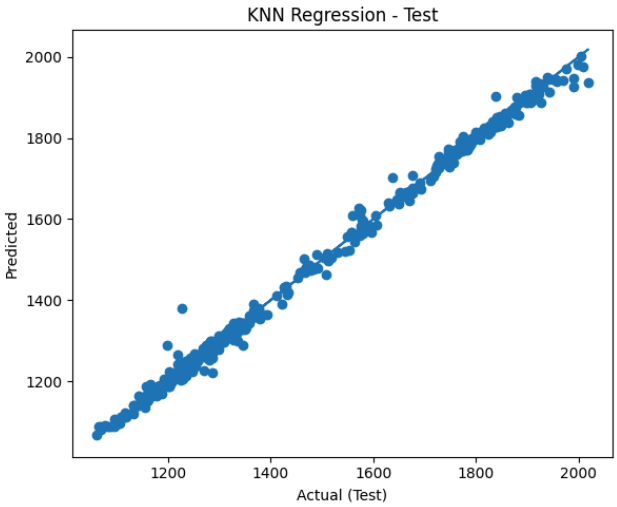
```
========================================
Ridge Regression PERFORMANCE
========================================

--- TRAINING METRICS ---
MAE: 2.975188169093759
MSE: 18.096199334244687
RMSE: 4.253962780072798
R2: 0.9997243247513536

--- TESTING METRICS ---
MAE: 2.8563152622484984
MSE: 17.24649727065702
RMSE: 4.152890230990583
R2: 0.9997467046561564
```



In [55]:
```
compare_train_test(best_knn, "KNN Regression", X_train, X_test, y_train, y_tes
```

```
========================================
KNN Regression PERFORMANCE
========================================

--- TRAINING METRICS ---
MAE: 7.158713223407693
MSE: 146.9216779825412
RMSE: 12.121125277074782
R2: 0.9977618134415257

--- TESTING METRICS ---
MAE: 10.012790697674422
MSE: 267.3016537467701
RMSE: 16.349362487472412
R2: 0.996074202011388
```


KNN Regression - Train
Predicted
Actual (Train)
1000
1200
1400
1600
1800
2000


KNN Regression - Test
Predicted
Actual (Test)
1200
1400
1600
1800
2000